

DTs-RFs-GB-XGB

Decision Trees

DTs are piecewise greedy linear classifiers that can handle missing data and work with heterogenous data types.

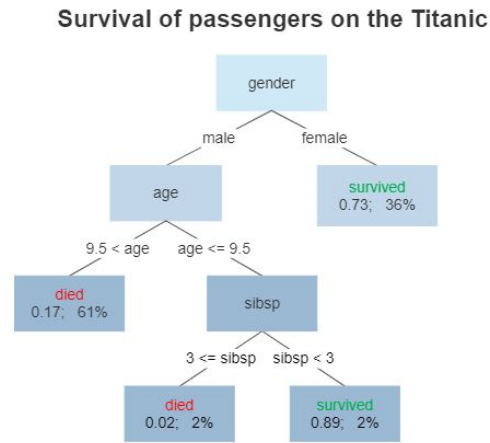


Figure 1: Simple DT for classifying survival on the Titanic.

Information Gain/ Entropy/ Gini Impurity

Decision criterions for choosing a feature to add to a DT.

Entropy

Entropy quantifies the uncertainty or disorder in a dataset. For a dataset with classes $c_1, c_2, \dots, c_k$ and probabilities p_i :

$$\text{Entropy}(S) = -\sum_{i=1}^k p_i \log_2(p_i)$$

where p_i is proportion of samples in class i and Entropy being 0 indicates a “pure” node (all one class) and Entropy being 1 indicates maximum uncertainty.

Information Gain

Measures reduction in entropy when data is split on a feature. Used to select the best feature to split on.

$$\text{Information Gain}(S, A) = \text{Entropy}(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \cdot \text{Entropy}(S_v)$$

where S is the dataset, A is a feature, and S_v is the subset of S where $A = v$.

Gini Impurity

Measures the probability of misclassifying a random instance.

$$\text{Gini}(S) = 1 - \sum_{i=1}^k p_i^2$$

where p_i is the proportion of class i in node S ; $\text{Gini} = 0$ implies a pure node.

NOTES:

1. The total Entropy/ InfoGain/ Gini Impurity is a weighted sum over all child nodes.
2. Gini Impurity was used in early versions of DTs. Now, most algorithms stick to Entropy + InfoGain.
3. Low Gini Impurity is good for splitting. High InfoGain is good for splitting.
4. Entropy is used to compute InfoGain. They're not separate criterions.
5. Continuous features can be split on a binary threshold based on highest InfoGain.
6. We can set thresholds on when we allow splits to occur; for example 50/50 splits are bad.
7. Missing feature values can be imputed with either the mode (for categorical features), or the mean/ median (for continuous features), or using another feature which has high correlation with this feature.

The above discussion generally applies to Classification DTs.

Regression Trees

- Similar to classification trees. Split criterion is either MSE or MAE instead of InfoGain. Leaf output is mean or median instead of mode. Evaluation is MSE/ RMSE/ MAE instead of accuracy/ precision/ recall. DTs can include both classification and regression, though. Both splitting criterion and final evaluation will be weighted averages of classification and regression metrics.
- Splitting criterion for continuous features is a bit more annoying. Instead of checking residuals (MAE/MSE) for each threshold between two possible threshold values, consider binary search. This only works for convex residual functions.
- Regression trees where feature values (and consequently, thresholds) can take on a high number of possible values are prone to overfitting (low bias + high variance). Consider splitting based on requiring a minimum number of observed instances.
- In the case of multiple features, start with the feature sub-tree whose root gives the lowest residuals.

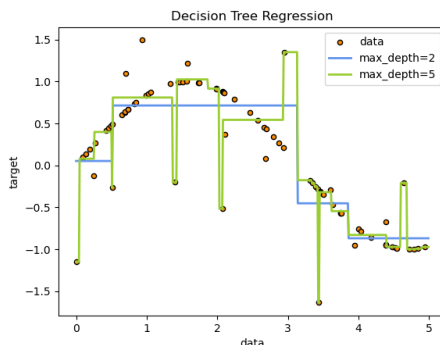


Figure 2: $\text{DT}_{\text{depth}=3}$ overfits. $\text{DT}_{\text{depth}=2}$ learns the major tendencies in the data and does not overfit.

Pruning Regression Trees - Cost Complexity Pruning

Prune sub-trees into single nodes to avoid overfitting with overly complex rules. Pruned trees will have higher residuals on training data, but are likely to generalize better.

Given K candidates, C_k , derived by iteratively pruning an overfit tree, a TreeScore can be computed as: $\text{TreeScore} = \text{SSR} + \alpha T$ where SSR is the sum of squared residuals, T is the number of leaf nodes, and α is a hyperparameter tuned through cross-validation. The candidate C_k with the lowest TreeScore is used for predictions.

Random Forests

DTs are easy to build, easy to use, and easy to interpret. But accuracy is an issue; they overfit to the training data but are not flexible when generalizing to unseen data. Random Forests (RFs) combine the simplicity of DTs with flexibility resulting in a vast improvement in generalization ability.

Building Random Forests (Bagging)

Steps to convert a DT into an RF:

1. Create a bootstrapped dataset: randomly subsample (with replacement) from the training data.
2. Build a DT using a random subset of features from the bootstrapped dataset. The feature sampling occurs at each step of building the DT.
3. Go to #1. Repeat to generate multiple DTs from subsets of boths features and data.
4. During inference, each DT can vote (classification) for, or contribute to the mean (regression) of, the final aggregated prediction.

This method of bootstrapping to build DTs and aggregating predictions is known as BAGging.

NOTES

1. Bootstrapping (sampling with replacement) will cause some samples to never be chosen. These **out of bag** samples can be used to measure the generalization ability of the RF.
2. The **out of bag** samples can also be used to tune hyperparameters such as the probability of sampling with replacement, and the number of features sampled for building the individual DTs.

Handling Missing Data

Missing data can occur both during learning and during inference.

Missing data during learning

Make an initial guess and iteratively refine it until we (hopefully) arrive at a better guess.

- Initial guess heuristic: for a missing feature value (x_{ij}), filter the entire by its label value (Y_i) and the initial guess becomes the mode (categorical)/ median (continuous) of filtered feature values (x_{kj}) for all $Y_i == Y_k$.
- With initial guesses filled in, build an RF.
- Build a proximity matrix P (sample similarity matrix in the RF observation space): Run inference on all samples using all the DTs. Every sample that ends at a leaf node of a DT is similar to any another sample ending at the same leaf node of a DT. Use these proximities to populate a proximity matrix for all samples over all DTs and normalize it to $[0, 1]$.
- For any missing feature (x_{ij} with j^{th} missing feature for sample x_i), compute weighted voting (classification)/ mean (regression) as follows:
 - Let M_j be the index set of samples with observed feature j : $M_j = \{i \in \{1, \dots, N\} : x_{ij} \text{ is observed}\}$.
 - The standard proximity weighted imputation for missing x_{ij} is: $\hat{x}_{ij} = \frac{\sum_{l \in M_j} P_{il} x_{lj}}{\sum_{l \in M_j} P_{il}}$, where x_{lj} contributes proportionally to how “close” sample l is to sample i under the forest’s learned structure.
 - The aggregation becomes voting instead of summation for classification tasks.

Similarity/ Sample Clustering

- The proximity matrix can be used as a similarity matrix in the RFs output space and used for measuring similarity/ clustering/ visualization.
- The proximity matrix can be negated to form a distance matrix. This allows us to work with heterogeneous data and arrive at a distance matrix in the RFs output space.

Missing data during inference

- Create duplicates of the sample with missing feature and label with all possibilities.
- Run through the RF and use the above method to aggregate and find the most likely value of the missing feature based on the candidate which results in being closest to the label possibilities.

AdaBoost (in a hurry)

AdaBoost (Adaptive Boosting) is a meta-algorithm that combines many weak learners — typically decision stumps — into a single strong classifier by iteratively reweighting training examples so that each new learner focuses on the mistakes of its predecessors.

How it works

- Init sample weights: Assign each of the N training examples a weight $w_{i,1} = 1/N$.
- Iterate through trees $t = 1, \dots, T$:
 - Train a weak learner $h_t(x)$ to minimize the weighted error: $\epsilon_t = \sum_{i=1}^N w_{i,t} \mathbf{1}\{h_t(x_i) \neq y_i\}$.
 - Compute its importance: $\alpha_t = \frac{1}{2} \ln\left(\frac{1-\epsilon_t}{\epsilon_t}\right)$.
 - Update weights as: $w_{i,t+1} = \frac{w_{i,t} \exp(-\alpha_t y_i h_t(x_i))}{\sum_j w_{j,t} \exp(-\alpha_t y_j h_t(x_j))}$.
- Combine weak learners: $H(x) = \text{sign}(\sum_{t=1}^T \alpha_t h_t(x))$.

NOTE:

1. AdaBoost builds an additive model while greedily minimizing the exponential loss.
2. The weak learners are usually stumps (DTs with one root and two leaves), but can be any weak estimator.
3. While often robust to overfitting (even as T grows), AdaBoost is sensitive to noisy labels and outliers, since misclassified points gain ever-increasing weight.

Gradient Boosting

Gradient Boosting (GB) is an ensemble technique that builds a prediction model in a stage-wise fashion by sequentially fitting weak learners to the negative gradient of a chosen loss function enabling direct optimization of arbitrary differentiable losses. Rather than reweighting samples (like AdaBoost), GB fits each new learner to the negative gradient (pseudo-residual) of the loss function with respect to the current model's predictions.

Algorithm

(Apologies for the math that follows. But, it's really interesting to see how DTs have gone from piecewise linear functions to now performing gradient descent!)

- GB constructs an additive model: $H_M(x) = \sum_{m=1}^M \gamma_m h_m(x)$ where each h_m is a weak learner fit to correct the errors of $h_1, \dots, h_{m-1}$ and γ_m is a step size associated with the learners.
- At iteration m , GB finds h_m that minimizes the risk: $h_m = \arg \min_h \sum_{i=1}^n \ell(y_i, H_{m-1}(x_i) + h(x_i))$.
 - A first order Taylor expansion around $H_{m-1}(x_i)$ leads to fitting h_m around the gradients and gives us the pseudo-residual $r_{im} = - \left. \frac{\partial \ell(y_i, \hat{y})}{\partial \hat{y}} \right|_{\hat{y}=H_{m-1}(x_i)}$. Here, $\ell(y_i, \hat{y}_i)$ is the loss function (like MSE/ BCE) and $H_{m-1}(x_i)$ is the model's prediction at the previous iteration ($m-1$).
 - This negative gradient indicates the direction in which the model's predictions should be adjusted to minimize the loss function.
 - Simple example: If we use the squared error as a loss function for regression ($\ell(y, \hat{y}) = \frac{1}{2}(y - \hat{y})^2$), the derivative with respect to $\hat{y}$ is $\hat{y} - y$ and the pseudo-residual becomes $r_{im} = y_i - H_{m-1}(x_i)$. So the pseudo-residual is the actual residual.
- For $m = 1, \dots, M$:
 - Compute pseudo-residuals r_{im} .
 - Fit a regression tree $h_m(x)$ to $\{(x_i, r_{im})\}$.
 - Compute leaf-wise optimal step sizes γ_{jm} for each leaf R_{jm} :

$$\gamma_{jm} = \arg \min_{\gamma} \sum_{x_i \in R_{jm}} \ell(y_i, H_{m-1}(x_i) + \gamma)$$
 - Update:

$$H_m(x) = F_{m-1}(x) + \nu \sum_{j=1}^{J_m} \gamma_{jm} \mathbf{1}_{\{x \in R_{jm}\}}$$
 , where ν is the learning rate.

Regularization

- Shrinkage (Learning Rate): A factor $\nu \in (0, 1]$ multiplies each tree's contribution, slowing learning and improving generalization. Typical values are 0.01 – 0.1.
- Subsampling (Stochastic Gradient Boosting): At each iteration, fit trees on a random subsample of the data (without replacement). This reduces variance and speeds up training.
- Tree-specific Regularization:
 - Max depth / leaf count: Constrains tree complexity.
 - Minimum samples per leaf / split: Enforces a minimum number of samples.

Popular Implementations

- XGBoost
 - Second-order Taylor approximation: uses both the gradient and the hessian to compute updates.
 - Sparse optimization: handles missing values efficiently.
 - Out-of-core and distributed training.
- LightGBM
 - Histogram-based splitting: buckets continuous features into discrete bins.
 - Gradient-based One-Side Sampling (GOSS): prioritizes data points with large gradients.
 - Exclusive Feature Bundling (EFB): combines mutually exclusive features to reduce dimensionality.
- CatBoost
 - Ordered boosting: reduces target leakage when handling categorical features via permutation-driven training.
 - Symmetric trees: enforce balanced splits for faster, more stable training.
 - Native categorical encoding: automatically handles high-cardinality features

All three are popular and can give slight edges on different benchmarks. Picking one and learning to tune it well is generally a good idea.

XGBoost

Gradient Boosting optimized over more than a decade.

Installation

```
pip install xgboost
```

API usage

```
import xgboost as xgb
from xgboost import XGBClassifier, XGBRegressor
model = XGBClassifier() # or XGBRegressor()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Core Components

Concept	Explanation
Booster	Core model that learns: 'gbtree' (default, trees) / 'dart' (dropout trees) / 'gblinear' (linear model)
Objective	Loss function you minimize ('binary:logistic' , 'multi:softprob' , 'reg:squarederror' , etc.)
Base Score	Initial prediction score before boosting (default 0.5 for classification)

Hyperparameters

Category	Parameter	Role	Typical Range
Model Complexity	max_depth	Max tree depth	3–12
	min_child_weight	Min sum of instance weight (hessian) needed in a child	1–10
	gamma	Min loss reduction to make a split	0–5
	lambda	L2 regularization	0–100
	alpha	L1 regularization	0–100
Training Dynamics	learning_rate (eta)	Step shrinkage	0.01–0.3
	n_estimators	Number of trees	100–10,000
	subsample	Row sampling per tree	0.5–1
	colsample_bytree	Feature sampling per tree	0.5–1
Imbalanced Data	scale_pos_weight	Balancing positive/negative classes	(#neg / #pos)
Tree Construction	grow_policy	How trees grow: 'depthwise' (default), 'lossguide' (for large datasets)	—

Rule of thumb:

- Decrease **learning_rate** → increase **n_estimators**.
- Deeper trees (**max_depth**) → more complex models, more overfitting.
- Use **early_stopping_rounds** when tuning to avoid overfitting.

Common Mistakes to Avoid

Mistake	How to Avoid
Setting <code>n_estimators</code> too high without <code>early_stopping</code>	Always combine with early stopping
Forgetting regularization (<code>lambda</code> , <code>alpha</code>)	Especially for deep trees
Not tuning <code>scale_pos_weight</code> for imbalanced classification	Always check class balance
Using default <code>learning_rate=0.3</code> for large datasets	Reduce to 0.05 or 0.01
Ignoring categorical features	XGBoost does not natively handle categoricals — you must one-hot encode or label encode

Feature Engineering/ Best Practices

See xgboosting.com for a superset of everything one might need.

References:

1. Decision and Classification Trees [statquest](#)
2. Random Forests [statquest](#)
3. Gradient Boosting [statquest](#)
4. XGBoost [statquest](#)
5. Decision Forests [google-dev](#)
6. Feature Engineering/ Best Practices xgboosting.com